

# MASTER NOTES: Stack & Queue – Implementation Basics

---

## Stack using Array

 **Problem:** Implement stack operations (push, pop, top, isEmpty) using a fixed-size array.

 **Pattern:** LIFO (Last In First Out).

 **Trick:**

Think of stack as a **plate pile**. Push = put new plate, Pop = remove top plate.

 **Variation:**

- Dynamic stack (using vector instead of fixed array).
- Implement 2 stacks in 1 array.

 **Code (C++):**

```
#include <bits/stdc++.h>
using namespace std;

class Stack {
    int top, capacity;
    int* arr;
public:
    Stack(int size) {
        capacity = size;
        arr = new int[size];
        top = -1;
    }
    void push(int x) {
        if (top == capacity - 1) {
            cout << "Overflow\n"; return;
        }
        arr[++top] = x;
    }
    void pop() {
        if (top == -1) {
            cout << "Underflow\n"; return;
        }
        top--;
    }
    int peek() {
        return (top == -1) ? -1 : arr[top];
    }
};
```

```

    }
    bool empty() {
        return top == -1;
    }
};

int main() {
    Stack st(5);
    st.push(10); st.push(20); st.push(30);
    cout << st.peek(); // 30
}

```

 **TC:**  $O(1)$  for push/pop/peek.

 **SC:**  $O(n)$ .

### JAVA:

```

class Stack {
    private int top, capacity;
    private int[] arr;

    // Constructor
    Stack(int size) {
        capacity = size;
        arr = new int[size];
        top = -1;
    }

    // Push operation
    void push(int x) {
        if (top == capacity - 1) {
            System.out.println("Overflow");
            return;
        }
        arr[++top] = x;
    }

    // Pop operation
    void pop() {
        if (top == -1) {
            System.out.println("Underflow");
            return;
        }
        top--;
    }

    // Peek operation
    int peek() {
        return (top == -1) ? -1 : arr[top];
    }
}

```

```

    }

    // Empty check
    boolean isEmpty() {
        return top == -1;
    }
}

public class Main {
    public static void main(String[] args) {
        Stack st = new Stack(5);
        st.push(10);
        st.push(20);
        st.push(30);
        System.out.println(st.peek()); // 30
    }
}

```

---

## 2 Queue using Array

 **Problem:** Implement queue operations (enqueue, dequeue, front, isEmpty) using an array.

 **Pattern: FIFO (First In First Out).**

 **Trick:**  
Think of **people in a line** → first person gets served first.

 **Variation:**

- Circular Queue (to reuse space).
- Double-ended Queue (Deque).

 **Code (Circular Queue):**

```

class Queue {
    int *arr, front, rear, size, capacity;
public:
    Queue(int c) {
        capacity = c; size = 0;
        arr = new int[c]; front = 0; rear = -1;
    }
    void enqueue(int x) {
        if (size == capacity) { cout << "Full\n"; return; }
        rear = (rear + 1) % capacity;
        arr[rear] = x;
        size++;
    }
}

```

```

void dequeue() {
    if (size == 0) { cout << "Empty\n"; return; }
    front = (front + 1) % capacity;
    size--;
}
int getFront() { return (size == 0) ? -1 : arr[front]; }
bool empty() { return size == 0; }
};

```

 **TC:**  $O(1)$ .

 **SC:**  $O(n)$ .

### 3 Stack using Linked List

 **Problem:** Implement stack with dynamic memory (no fixed size).

 **Pattern:** Linked List Head = Top of stack.

 **Trick:**

Push = Insert at head. Pop = Remove from head.

 **Variation:**

- Dynamic memory handling.
- Implement Min Stack using linked list.

 **Code:**

```

struct Node {
    int data;
    Node* next;
    Node(int x): data(x), next(NULL) {}
};

class Stack {
    Node* top;
public:
    Stack() { top = NULL; }
    void push(int x) {
        Node* temp = new Node(x);
        temp->next = top;
        top = temp;
    }
    void pop() {
        if (!top) { cout << "Underflow\n"; return; }
        Node* temp = top;
        top = top->next;
        delete temp;
    }
};

```

```

    }
    int peek() { return top ? top->data : -1; }
    bool empty() { return top == NULL; }
};

```

 **TC:**  $O(1)$ .

 **SC:**  $O(n)$ .

## 4 Queue using Linked List

 **Problem:** Implement queue using linked list (dynamic memory).

 **Pattern:** Head = Front, Tail = Rear.

 **Trick:**  
 Enqueue  $\rightarrow$  insert at tail. Dequeue  $\rightarrow$  delete at head.

 **Variation:**

- Dequeue using doubly linked list.

 **Code:**

```

struct Node {
    int data;
    Node* next;
    Node(int x): data(x), next(NULL) {}
};

class Queue {
    Node* front; Node* rear;
public:
    Queue() { front = rear = NULL; }
    void enqueue(int x) {
        Node* temp = new Node(x);
        if (!rear) { front = rear = temp; return; }
        rear->next = temp;
        rear = temp;
    }
    void dequeue() {
        if (!front) { cout << "Empty\n"; return; }
        Node* temp = front;
        front = front->next;
        if (!front) rear = NULL;
        delete temp;
    }
    int getFront() { return front ? front->data : -1; }
    bool empty() { return front == NULL; }
};

```

```
};
```

 **TC:**  $O(1)$ .

 **SC:**  $O(n)$ .

---

## 5 Stack using Queue

 **Problem:** Implement stack using queue(s).

 **Pattern:** Make one operation costly.

 **Trick:**

- Make **push costly** → rotate queue until new element comes front.
- Make **pop costly** → transfer elements except last to helper queue.

 **Variation:**

- Use 1 queue vs 2 queues.

 **Code (1 queue, costly push):**

```
class Stack {
    queue<int> q;
public:
    void push(int x) {
        q.push(x);
        for (int i = 0; i < (int)q.size()-1; i++) {
            q.push(q.front());
            q.pop();
        }
    }
    void pop() { if(!q.empty()) q.pop(); }
    int top() { return q.empty() ? -1 : q.front(); }
    bool empty() { return q.empty(); }
};
```

 **Push:**  $O(n)$ , **Pop:**  $O(1)$ .

 **SC:**  $O(n)$ .

---

## 6 Queue using Stack

📌 **Problem:** Implement queue using stack(s).

✂️ **Pattern:** Make one operation costly.

🎯 **Trick:**

- Use two stacks ( $s_1, s_2$ ).
- Enqueue = push to  $s_1$ .
- Dequeue = transfer all from  $s_1 \rightarrow s_2$  when  $s_2$  empty.

🔄 **Variation:**

- Lazy transfer (only when needed).

⚡ **Code:**

```
class Queue {
    stack<int> s1, s2;
public:
    void enqueue(int x) { s1.push(x); }
    void dequeue() {
        if (s2.empty()) {
            while (!s1.empty()) {
                s2.push(s1.top()); s1.pop();
            }
        }
        if (!s2.empty()) s2.pop();
    }
    int front() {
        if (s2.empty()) {
            while (!s1.empty()) {
                s2.push(s1.top()); s1.pop();
            }
        }
        return s2.empty() ? -1 : s2.top();
    }
    bool empty() { return s1.empty() && s2.empty(); }
};
```

🕒 **Amortized TC:**  $O(1)$ . Worst  $O(n)$ .

📄 **SC:**  $O(n)$ .



## Recap (Memory Hook)

- **Stack = Plates (LIFO)**
- **Queue = Line (FIFO)**

- **LinkedList version** → head/tail trick.
- **X using Y** → always trade-off:
  - Stack↔Queue: Make one operation costly.
  - Queue↔Stack: Use 2 structures.

# 1 Undo/Redo in Text Editor

## ◆ Idea

- **Two stacks:**
  - `undoStack`: keeps track of performed actions.
  - `redoStack`: keeps track of undone actions.
- **When user types/deletes:**
  - Push action onto `undoStack`.
  - Clear `redoStack` (since redo only applies after an undo).
- **Undo:**
  - Pop from `undoStack` → push into `redoStack`.
  - Reverse that action in editor.
- **Redo:**
  - Pop from `redoStack` → push into `undoStack`.
  - Apply that action again.

---

## ◆ Java Example

```
import java.util.Stack;

class TextEditor {
    private StringBuilder text;
    private Stack<String> undoStack, redoStack;

    public TextEditor() {
        text = new StringBuilder();
        undoStack = new Stack<>();
        redoStack = new Stack<>();
    }

    public void type(String word) {
        text.append(word);
        undoStack.push(word); // store action
        redoStack.clear();    // new action clears redo
    }

    public void undo() {
        if (!undoStack.isEmpty()) {
            String last = undoStack.pop();
            int len = last.length();
            text.delete(text.length() - len, text.length());
            redoStack.push(last); // store undone action
        }
    }
}
```

```

    }
}

public void redo() {
    if (!redoStack.isEmpty()) {
        String last = redoStack.pop();
        text.append(last);
        undoStack.push(last);
    }
}

public String getText() {
    return text.toString();
}
}

public class Main {
    public static void main(String[] args) {
        TextEditor editor = new TextEditor();
        editor.type("Hello ");
        editor.type("World");
        System.out.println(editor.getText()); // Hello World

        editor.undo();
        System.out.println(editor.getText()); // Hello

        editor.redo();
        System.out.println(editor.getText()); // Hello World
    }
}

```

---

## 2 Balanced HTML/XML Tags Validation

### ◆ Idea

- Push **opening tags** onto a stack.
  - When you see a **closing tag**, check if it matches the top of the stack.
  - If mismatch → invalid.
  - After traversal, if stack not empty → invalid.
- 

### ◆ Java Example

```

import java.util.Stack;

class HTMLValidator {
    public static boolean isValid(String code) {
        Stack<String> stack = new Stack<>();
        int i = 0;
        while (i < code.length()) {
            if (code.charAt(i) == '<') {

```

```

        int j = code.indexOf(">", i);
        if (j == -1) return false;
        String tag = code.substring(i + 1, j);

        if (!tag.startsWith("/")) { // opening tag
            stack.push(tag);
        } else { // closing tag
            if (stack.isEmpty() ||
!stack.peek().equals(tag.substring(1)))
                return false;
            stack.pop();
        }
        i = j + 1;
    } else {
        i++;
    }
}
return stack.isEmpty();
}

public static void main(String[] args) {
    System.out.println(isValid("<html><body></body></html>")); // true
    System.out.println(isValid("<div><p></div></p>")); // false
}
}

```

---

## Similar LeetCode Questions

### Undo/Redo (Editor-like)

- LeetCode **1598. Crawler Log Folder** → behaves like undo/redo (./ undo, ./ stay, <dir> like typing).
- LeetCode **1209. Remove All Adjacent Duplicates in String II** → mimics undo/redo on string states.

### Balanced HTML/XML Tags

- LeetCode **20. Valid Parentheses** → exactly same concept with brackets.
- LeetCode **591. Tag Validator** → full HTML/XML-like tag validation (hard).

## MASTER NOTES: Expression Problems (Stack Pattern)

## 1 Balanced Parentheses

 **Problem:** Given a string of brackets `() [] {}`, check if it is balanced.

 **Pattern:** Stack keeps track of **opening brackets**.

 **Trick:**

- **Push opening,**
- **Pop when closing,**
- If mismatch  $\rightarrow$  invalid.  
 Think: *matching socks in a laundry basket.*

 **Variation:**

- Multiple bracket types `{}, [], ()`.
- Minimum add/remove to balance parentheses.

 **Code (C++):**

```
bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c=='(' || c=='[' || c=='{') st.push(c);
        else {
            if (st.empty()) return false;
            char t = st.top(); st.pop();
            if ((c==')' && t!='(') ||
                (c==']' && t!='[') ||
                (c=='}' && t!='{')) return false;
        }
    }
    return st.empty();
}
```

 **TC:**  $O(n)$ ,  **SC:**  $O(n)$ .

---

## 2 Infix $\rightarrow$ Postfix Conversion

 **Problem:** Convert `"A+B*C"`  $\rightarrow$  `"ABC*+"`.

 **Pattern:** Use stack for **operators**, append operands to result.

## Trick:

- Operand  $\rightarrow$  add to result.
  - Operator  $\rightarrow$  pop higher/equal precedence before pushing.
  - (  $\rightarrow$  push.
  - )  $\rightarrow$  pop till (.
- 👉 Think: operators wait in stack until they deserve priority.

## Operator Precedence and Associativity Table

| Operator(s) | Description               | Precedence  | Associativity                                                                                   |
|-------------|---------------------------|-------------|-------------------------------------------------------------------------------------------------|
| $\wedge$    | Exponentiation (power)    | 3 (Highest) | Right to Left  |
| $*, /$      | Multiplication / Division | 2           | Left to Right  |
| $+, -$      | Addition / Subtraction    | 1 (Lowest)  | Left to Right  |

 **Higher precedence** = evaluated **earlier**

 **Associativity** = which direction to evaluate when **same precedence** operators appear

---

## Examples

| Expression            | Evaluation Order                     | Reason                               |
|-----------------------|--------------------------------------|--------------------------------------|
| $A + B * C$           | $B * C$ first $\rightarrow$ then $+$ | $*$ has higher precedence            |
| $A * B / C$           | $(A * B) / C$                        | same precedence, left associative    |
| $A \wedge B \wedge C$ | $A \wedge (B \wedge C)$              | $\wedge$ is <b>right associative</b> |
| $A - B - C$           | $(A - B) - C$                        | $-$ is left associative              |

**Infix:**  $A + B * C - D / E$

**Steps:**

- $A \rightarrow$  output
- $+$   $\rightarrow$  push
- $B \rightarrow$  output
- $*$   $\rightarrow$  push (higher than +)
- $C \rightarrow$  output

- $- \rightarrow \text{stack top} * (\text{higher}) \rightarrow \text{pop} * \rightarrow \text{output}$   
Now  $+$  (equal)  $\rightarrow \text{pop} + \rightarrow \text{output}$   
Push  $-$
- $D \rightarrow \text{output}$
- $/ \rightarrow \text{push (higher than } -)$
- $E \rightarrow \text{output}$   
End  $\rightarrow \text{pop } / , -$   
**Postfix:** A B C \* + D E / -

#### ✓ Example 7: Complex parentheses

**Infix:** A + (B \* C - (D / E ^ F) \* G) \* H

Postfix:

**Postfix:**

ABC\*DEF^/G\*-H\*+

 **Variation:** Handle multi-digit numbers, variables, functions.

 **Code:**

```
int prec(char c) {
    if (c=='^') return 3;
    if (c=='*' || c=='/') return 2;
    if (c=='+' || c=='-') return 1;
    return -1;
}

string infixToPostfix(string s) {
    stack<char> st; string res="";
    for (char c: s) {
        if (isalnum(c)) res += c;
        else if (c=='(') st.push(c);
        else if (c==')') {
            while (!st.empty() && st.top()!='(') {
                res += st.top(); st.pop();
            }
            st.pop();
        } else {
            while (!st.empty() && prec(st.top())>=prec(c)) {
                res += st.top(); st.pop();
            }
            st.push(c);
        }
    }
    while (!st.empty()) { res += st.top(); st.pop(); }
    return res;
}
```

### ✓ Equivalent Java Code

```
import java.util.*;

public class InfixToPostfix {
    static int prec(char c) {
        return (c == '^') ? 3 : (c == '*' || c == '/') ? 2 : (c == '+' || c
== '-') ? 1 : 0;
    }

    static String convert(String exp) {
        Stack<Character> st = new Stack<>();
        StringBuilder res = new StringBuilder();

        for (char c : exp.toCharArray()) {
            if (Character.isLetterOrDigit(c)) res.append(c);
            else if (c == '(') st.push(c);
            else if (c == ')') {
                while (!st.isEmpty() && st.peek() != '(')
res.append(st.pop());
                st.pop(); // remove '('
            } else { // operator
                while (!st.isEmpty() && prec(st.peek()) >= prec(c)) {
                    if (c == '^' && st.peek() == '^') break; // right-
associative
                    res.append(st.pop());
                }
                st.push(c);
            }
        }
        while (!st.isEmpty()) res.append(st.pop());
        return res.toString();
    }

    public static void main(String[] args) {
        String exp = "A+(B*C-(D/E^F)*G)*H";
        System.out.println("Postfix: " + convert(exp));
    }
}
```

🕒 TC: O(n), 📄 SC: O(n).

---

## 3 Infix → Prefix Conversion

📌 **Problem:** "A+B\*C" → "+A\*BC".

✂️ **Pattern:** Trick = Reverse + Postfix.

🎯 **Trick:**

1. Reverse infix.

2. Swap ( and ).
  3. Convert to postfix.
  4. Reverse result → Prefix.
- 👉 Prefix = *reversed postfix*.

⚡ **Code** (reuse above function):

```
string infixToPrefix(string s) {
    reverse(s.begin(), s.end());
    for (char &c: s) {
        if (c=='(') c=')';
        else if (c==')') c='(';
    }
    string post = infixToPostfix(s);
    reverse(post.begin(), post.end());
    return post;
}
```

**Java code:**

```
static String infixToPrefix(String s) {
    StringBuilder sb = new StringBuilder(s).reverse();
    for (int i = 0; i < sb.length(); i++) {
        if (sb.charAt(i) == '(') sb.setCharAt(i, ')');
        else if (sb.charAt(i) == ')') sb.setCharAt(i, '(');
    }
    String post = infixToPostfix(sb.toString());
    return new StringBuilder(post).reverse().toString();
}
```

---

## 4 Postfix → Infix Conversion

📌 **Problem:** "ABC\*+" → "A+B\*C".

✖ **Pattern:** Stack stores **strings of operands/expressions**.

🎯 **Trick:**

- Operand → push string.
  - Operator → pop 2, combine "(" + a + op + b + ")", push back.
- 👉 Think: *build tree bottom-up*.

### ⚡ Code:

```
string postfixToInfix(string s) {
    stack<string> st;
    for (char c: s) {
        if (isalnum(c)) st.push(string(1,c));
        else {
            string b = st.top(); st.pop();
            string a = st.top(); st.pop();
            st.push("(" + a + c + b + ")");
        }
    }
    return st.top();
}
```

### ✓ Short Java Code — postfixToInfix()

```
import java.util.*;

public class PostfixToInfix {
    public static void main(String[] args) {
        String exp = "ABC*+DE/-"; // Example postfix
        System.out.println("Infix: " + postfixToInfix(exp));
    }

    static String postfixToInfix(String s) {
        Stack<String> st = new Stack<>();
        for (char c : s.toCharArray()) {
            if (Character.isLetterOrDigit(c)) {
                st.push(String.valueOf(c));
            } else {
                String b = st.pop();
                String a = st.pop();
                st.push("(" + a + c + b + ")");
            }
        }
        return st.peek();
    }
}
```

---

## 5 Postfix → Prefix Conversion

📌 **Problem:** "ABC\*+" → "+A\*BC".

✖ **Pattern:** Similar to above, but combine as op + a + b.

🎯 **Trick:**

- Operand → push.

- Operator  $\rightarrow$  pop 2, push "op + a + b".

### ⚡ Code:

```
string postfixToPrefix(string s) {
    stack<string> st;
    for (char c: s) {
        if (isalnum(c)) st.push(string(1,c));
        else {
            string b = st.top(); st.pop();
            string a = st.top(); st.pop();
            st.push(c + a + b);
        }
    }
    return st.top();
}
```

Java:

```
static String postfixToPrefix(String s) {
    Stack<String> st = new Stack<>();
    for (char c : s.toCharArray()) {
        if (Character.isLetterOrDigit(c)) {
            st.push(String.valueOf(c));
        } else {
            String b = st.pop();
            String a = st.pop();
            st.push(c + a + b);
        }
    }
    return st.peek();
}
```

---

## 6 Prefix $\rightarrow$ Infix Conversion

📌 **Problem:** "+A\*BC"  $\rightarrow$  "A+B\*C".

✂ **Pattern:** Scan right  $\rightarrow$  left.

🎯 **Trick:**

- Operand  $\rightarrow$  push.
- Operator  $\rightarrow$  pop 2, combine "(" + a + op + b + ")".

### ⚡ Code:

```
string prefixToInfix(string s) {
```

```

stack<string> st;
for (int i=s.size()-1; i>=0; i--) {
    char c=s[i];
    if (isalnum(c)) st.push(string(1,c));
    else {
        string a = st.top(); st.pop();
        string b = st.top(); st.pop();
        st.push("(" + a + c + b + ")");
    }
}
return st.top();
}

```

---

## 7 Prefix → Postfix Conversion

 **Problem:** "+A\*BC" → "ABC\*+".

 **Pattern:** Scan right → left, build postfix.

 **Trick:**

- Operand → push.
- Operator → pop 2, push "a+b+op" in postfix style.

 **Code:**

```

string prefixToPostfix(string s) {
    stack<string> st;
    for (int i=s.size()-1; i>=0; i--) {
        char c = s[i];
        if (isalnum(c)) st.push(string(1,c));
        else {
            string a = st.top(); st.pop();
            string b = st.top(); st.pop();
            st.push(a + b + c);
        }
    }
    return st.top();
}

```

---

## 8 Expression Evaluation (Bonus Variation)

 **Problem:** Evaluate postfix "23\*54\*+9-".

 **Pattern:** Stack of **integers**.

## Trick:

- Operand  $\rightarrow$  push int.
  - Operator  $\rightarrow$  pop 2, apply, push result.
  - **evaluate** the postfix expression:
  - $23*54*+9-$
  - We'll do it **step-by-step** using a **stack**.
- 

-  Postfix Expression:

- $2\ 3\ *\ 5\ 4\ *\ +\ 9\ -$
- 

-  Evaluation Process:

| Step | Symbol | Action                                                         | Stack     |
|------|--------|----------------------------------------------------------------|-----------|
| 1    | 2      | Push operand                                                   | [2]       |
| 2    | 3      | Push operand                                                   | [2, 3]    |
| 3    | *      | Pop 3, 2 $\rightarrow 2 \times 3 = 6 \rightarrow$ Push result  | [6]       |
| 4    | 5      | Push operand                                                   | [6, 5]    |
| 5    | 4      | Push operand                                                   | [6, 5, 4] |
| 6    | *      | Pop 4, 5 $\rightarrow 5 \times 4 = 20 \rightarrow$ Push result | [6, 20]   |
| 7    | +      | Pop 20, 6 $\rightarrow 6 + 20 = 26 \rightarrow$ Push result    | [26]      |
| 8    | 9      | Push operand                                                   | [26, 9]   |
| 9    | -      | Pop 9, 26 $\rightarrow 26 - 9 = 17 \rightarrow$ Push result    | [17]      |

-  Final Answer:

- 17
- 

## Code:

```
int evalPostfix(string s) {
    stack<int> st;
    for (char c: s) {
        if (isdigit(c)) st.push(c-'0');
        else {
            int b=st.top(); st.pop();
```

```

        int a=st.top(); st.pop();
        if (c=='+') st.push(a+b);
        else if (c=='-') st.push(a-b);
        else if (c=='*') st.push(a*b);
        else if (c=='/') st.push(a/b);
    }
}
return st.top();
}

```

---



## Recap (Memory Hooks)

- **Balanced Brackets** → Socks pair rule.
- **Infix** → **Postfix** → "Operands walk straight, Operators wait in stack".
- **Infix** → **Prefix** → Reverse + Postfix + Reverse.
- **Postfix/Prefix Conversions** → "Pop 2, combine, push back".
- **Evaluation** → Same as conversion, but do math.



## 3. Next Greater / Smaller Element Family (Monotonic Stack)

---

### 1. Next Greater Element (NGE)

#### Problem:

For each element in array, find the **next greater element** to its right. If none exists, return -1.

#### Pattern:

- Use a **monotonic decreasing stack** (store indices or elements).
- Pop until you find a greater element, then assign.

#### Trick to remember:



“Think future: keep only bigger guys alive.”

#### Code (O(n)):

```

vector<int> nextGreaterElements(vector<int>& arr) {
    int n = arr.size();
    vector<int> ans(n, -1);

```

```

stack<int> st; // store indices
for (int i = 0; i < n; i++) {
    while (!st.empty() && arr[st.top()] < arr[i]) {
        ans[st.top()] = arr[i];
        st.pop();
    }
    st.push(i);
}
return ans;
}

```

**TC:**  $O(n)$ , **SC:**  $O(n)$ .

**Variation:**

- “Next Smaller Element”  $\rightarrow$  just reverse the comparison ( $>$ ).
- Works in stock prices, weather, building heights.

## 2. Next Greater Element II (Circular Array)

**Problem:**

Same as NGE but array is **circular**.

**Trick:**

👉 Scan array **twice** using  $i \% n$ .

**Code:**

```

vector<int> nextGreaterElementsII(vector<int>& arr) {
    int n = arr.size();
    vector<int> ans(n, -1);
    stack<int> st;
    for (int i = 0; i < 2*n; i++) {
        int num = arr[i % n];
        while (!st.empty() && arr[st.top()] < num) {
            ans[st.top()] = num;
            st.pop();
        }
        if (i < n) st.push(i);
    }
    return ans;
}

```

## 3. Next Smaller Element (NSE)

**Problem:**

For each element, find the **next smaller element** to its right.

**Pattern:**

- **Monotonic increasing stack.**

**Trick:**

👉 “Think future: keep only smaller guys alive.”

**Code:**

```
vector<int> nextSmallerElements(vector<int>& arr) {
    int n = arr.size();
    vector<int> ans(n, -1);
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && arr[st.top()] > arr[i]) {
            ans[st.top()] = arr[i];
            st.pop();
        }
        st.push(i);
    }
    return ans;
}
```

---

## 4. Number of NGEs to Right

**Problem:**

For each element, count how many greater elements exist to its right.

**Pattern:**

- Modify NGE logic but instead of storing value, store **count**.

**Trick:**

👉 “Count how many times I pop before finding a bigger guy.”

**Code:**

```
vector<int> countNGE(vector<int>& arr) {
    int n = arr.size();
    vector<int> ans(n, 0);
    stack<int> st;
    for (int i = 0; i < n; i++) {
        int cnt = 0;
        while (!st.empty() && arr[st.top()] < arr[i]) {
            ans[st.top()] = 1; // at least one greater exists
            cnt++;
            st.pop();
        }
        st.push(i);
    }
    return ans;
}
```

```

        st.pop();
    }
    st.push(i);
}
return ans;
}

```

*(Variation-heavy, sometimes asked in tricky ways.)*

---

## 5. Stock Span Problem

### Problem:

For each day, find the number of consecutive days (including today) where price  $\leq$  today's price.

### Pattern:

- Like NGE but count distances.

### Trick:

👉 Span = current index - index of last greater element.

### Code:

```

vector<int> stockSpan(vector<int>& price) {
    int n = price.size();
    vector<int> span(n);
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && price[st.top()] <= price[i])
            st.pop();
        span[i] = st.empty() ? (i+1) : (i - st.top());
        st.push(i);
    }
    return span;
}

```

---



## Master Trick Summary

- NGE  $\rightarrow$  decreasing stack (pop smaller).
  - NSE  $\rightarrow$  increasing stack (pop bigger).
  - Circular NGE  $\rightarrow$  run loop  $2n$ .
  - Stock Span  $\rightarrow$  distance to last greater.
  - Next Warmer Day (Leetcode 739)  $\rightarrow$  NGE + store index differences.
-



## Common Variations

- Next Greater to Left (just scan from right→left).
- Histogram largest rectangle (NSE + stack).
- Trapping Rain Water (use NGE + NSE or 2-pointers).
- Daily Temperatures (Leetcode 739).
- Sum of subarray minimums (NSE variation).



## 4. Interval / Range Problems with Stacks



### Pattern Rulebook

- "Range" problems = find **previous smaller** / **next smaller** / **previous greater** / **next greater**
  - Once you know **boundaries**, you can compute **contribution** (area, sum, etc.)
  - **Histogram Trick** → Convert to nearest smaller left & right = rectangle limits.
  - **Water Trick** → Max left & right = trap.
  - **Subarray Trick** → Every element contributes to multiple subarrays bounded by smaller/greater.
- 



### Problem 1: Trapping Rainwater (Hard)

 Problem Statement:

Given an array `height[]` of size `n`, each element represents elevation at that index. Compute **how much water can be trapped after raining**.

**Example:**

```
height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
```

---

### 1 Brute Force Approach

**Idea:**

- For each bar, water trapped =  $\min(\text{max\_left}, \text{max\_right}) - \text{height}[i]$
- Compute **max left** and **max right** for every element →  $O(n^2)$

### Code (C++):

```
int trap(vector<int>& height) {
    int n = height.size(), water = 0;
    for(int i = 0; i < n; i++){
        int leftMax = 0, rightMax = 0;
        for(int j = 0; j <= i; j++) leftMax = max(leftMax, height[j]);
        for(int j = i; j < n; j++) rightMax = max(rightMax, height[j]);
        water += min(leftMax, rightMax) - height[i];
    }
    return water;
}
```

✓ **Time Complexity:**  $O(n^2)$

✓ **Space Complexity:**  $O(1)$

### Trick to remember:

“Check left & right for every bar, take min  $\rightarrow$  subtract height”

---

## 2 Optimized with Precomputed Arrays

### Idea:

- Precompute **maxLeft[i]** and **maxRight[i]** arrays
- Then water at  $i = \min(\text{maxLeft}[i], \text{maxRight}[i]) - \text{height}[i] \rightarrow O(n)$

### Code (C++):

```
int trap(vector<int>& height) {
    int n = height.size();
    if(n == 0) return 0;
    vector<int> leftMax(n), rightMax(n);
    leftMax[0] = height[0];
    for(int i=1; i<n; i++) leftMax[i] = max(leftMax[i-1], height[i]);

    rightMax[n-1] = height[n-1];
    for(int i=n-2; i>=0; i--) rightMax[i] = max(rightMax[i+1], height[i]);

    int water = 0;
    for(int i=0; i<n; i++) water += min(leftMax[i], rightMax[i]) - height[i];

    return water;
}
```

✓ **Time Complexity:**  $O(n)$

✓ **Space Complexity:**  $O(n)$

**Trick:** “Precompute the walls on left & right → avoid repeated loops”

**Trick:**

- Water at index =  $\min(\text{leftMax}, \text{rightMax}) - \text{height}[i]$
- Can solve with **2-pointer  $O(n)$**  OR with **stack  $O(n)$** .

**Code ( $O(n)$ ,  $O(1)$  space 2-pointer):**

```
int trap(vector<int>& h) {
    int n=h.size(), l=0, r=n-1, lmax=0, rmax=0, water=0;
    while(l<r){
        if(h[l]<h[r]){
            lmax=max(lmax,h[l]);
            water += lmax-h[l];
            l++;
        } else {
            rmax=max(rmax,h[r]);
            water += rmax-h[r];
            r--;
        }
    }
    return water;
}
```

**TC Example**

`h = [0,1,0,2,1,0,1,3,2,1,2,1]`

👉 Output = 6

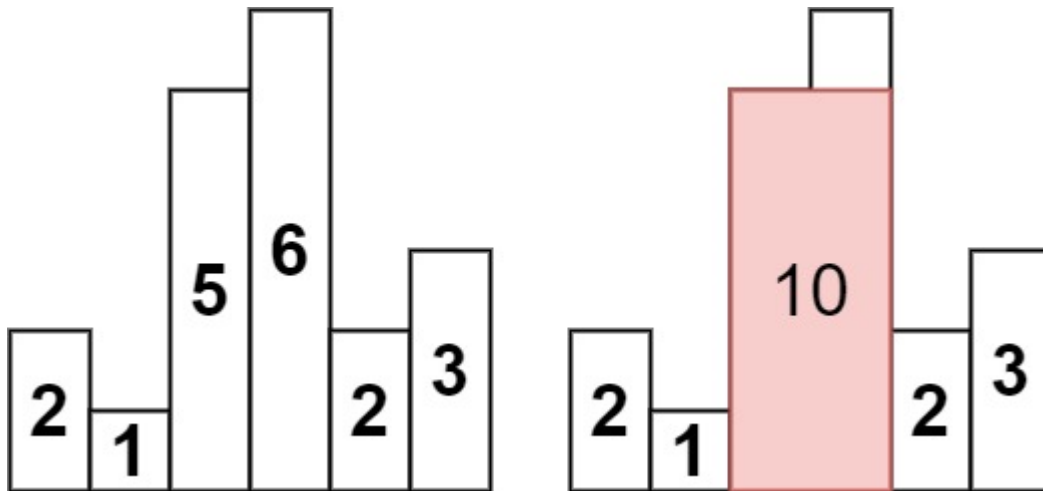
---

## Problem 2:

### 84. Largest Rectangle in Histogram

**Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return *the area of the largest rectangle in the histogram*.**

**Example 1:**



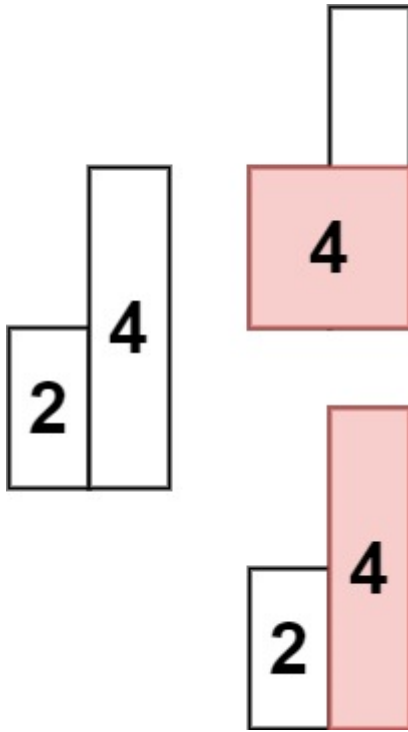
**Input: heights = [2,1,5,6,2,3]**

**Output: 10**

**Explanation: The above is a histogram where width of each bar is 1.**

**The largest rectangle is shown in the red area, which has an area = 10 units.**

**Example 2:**



**Input: heights = [2,4]**

**Output: 4**

**Constraints:**

- $1 \leq \text{heights.length} \leq 10^5$
- $0 \leq \text{heights}[i] \leq 10^4$

## **1) Prefix & Suffix Array Method ( $O(n)$ but with extra arrays)**

👉 Idea:

- For each bar  $i$ , find:
  - **Previous Smaller Element (PSE)** index  $\rightarrow$  nearest bar to the left smaller than  $h[i]$ .
  - **Next Smaller Element (NSE)** index  $\rightarrow$  nearest bar to the right smaller than  $h[i]$ .

- Then  $\text{width} = (\text{NSE}[i] - \text{PSE}[i] - 1)$ .
- $\text{Area} = h[i] * \text{width}$ .

---

### C++ Code (Prefix & Suffix Arrays)

```
int largestRectangleArea(vector<int>& h) {
    int n = h.size();
    vector<int> left(n), right(n);

    // Previous Smaller Element
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && h[st.top()] >= h[i]) st.pop();
        left[i] = st.empty() ? -1 : st.top();
        st.push(i);
    }

    // Next Smaller Element
    while (!st.empty()) st.pop();
    for (int i = n - 1; i >= 0; i--) {
        while (!st.empty() && h[st.top()] >= h[i]) st.pop();
        right[i] = st.empty() ? n : st.top();
        st.push(i);
    }

    // Compute max area
    int maxA = 0;
    for (int i = 0; i < n; i++) {
        int width = right[i] - left[i] - 1;
        maxA = max(maxA, h[i] * width);
    }
    return maxA;
}
```

---

## 2 Optimized Single Pass Stack Method ( $O(n)$ , $O(1)$ extra arrays)

👉 Trick:

- Traverse bars from  $0 \rightarrow n$ .
- Maintain stack of indices.
- When current bar is **smaller than stack top**, pop until valid.
- For each popped bar:
  - Height =  $h[\text{top}]$ .
  - Right = current index  $i$ .
  - Left = new stack top (after pop), or  $-1$  if empty.
  - Area = height \* (right - left - 1).

This avoids storing explicit prefix/suffix arrays.

---

### Optimized C++ Code

```
int largestRectangleArea(vector<int>& h) {
    int n = h.size();
    stack<int> st;
    int maxA = 0;

    for (int i = 0; i <= n; i++) {
        while (!st.empty() && (i == n || h[st.top()] >= h[i])) {
            int ht = h[st.top()];
            st.pop();
            int rt = i;
            int lt = st.empty() ? -1 : st.top();
            maxA = max(maxA, ht * (rt - lt - 1));
        }
        st.push(i);
    }
    return maxA;
}
```

---

### ✓ Summary:

- **Prefix + Suffix array approach** → easier to understand, uses two arrays, still  $O(n)$ .
- **Optimized single-pass stack approach** → no extra arrays, one loop with amortized  $O(n)$ .

📌 **Statement:** Given histogram bars, find max rectangle area.

### Trick:

- For each bar, find **previous smaller & next smaller** index → gives width.
- $\text{Area} = \text{height}[i] * (\text{right} - \text{left} - 1)$ .

### Code ( $O(n)$ ):

```
int largestRectangleArea(vector<int>& h) {
    int n=h.size();
    stack<int> st;
    int maxA=0;
    for(int i=0;i<=n;i++){
        while(!st.empty() && (i==n || h[st.top()]>=h[i])){
            int ht=h[st.top()]; st.pop();
            int rt=i;
            int lt=st.empty()?-1:st.top();
        }
        st.push(i);
    }
    return maxA;
}
```

```

        maxA=max(maxA, ht*(rt-lt-1));
    }
    st.push(i);
}
return maxA;
}

```

### TC Example

`h = [2,1,5,6,2,3]`

👉 Output = 10

## ◆ Algorithm Recap

For each index  $i$ :

- While stack not empty **and**  $h[\text{st.top}()] \geq h[i] \rightarrow \text{pop}$ .
- For popped height =  $h[\text{top}]$ :
  - $\text{rt} = i$  (first smaller to right).
  - $\text{lt} = \text{st.empty}() ? -1 : \text{st.top}()$  (first smaller to left).
  - $\text{Width} = \text{rt} - \text{lt} - 1$ .
  - $\text{Area} = \text{ht} * \text{width}$ .
- Push  $i$  on stack.

At the end, run  $i = n$  (sentinel smaller height).

---

## ◆ Dry Run on `[2,1,5,6,2,3]`

Initial

- $n = 6, \text{st} = [], \text{maxA} = 0$
- 

$i = 0 \rightarrow h[0] = 2$

- Stack empty  $\rightarrow$  push 0
  - $\text{st} = [0]$
-

$i = 1 \rightarrow h[1] = 1$

- $h[\text{st.top}()] = h[0] = 2 \geq 1 \rightarrow \text{pop } 0$ 
    - $ht = 2, rt = 1, lt = -1 \rightarrow \text{width} = 1 - (-1) - 1 = 1$
    - $\text{area} = 2 \times 1 = 2 \rightarrow \text{maxA} = 2$
  - Stack empty  $\rightarrow$  push 1
  - $\text{st} = [1]$
- 

$i = 2 \rightarrow h[2] = 5$

- $h[1] = 1 < 5 \rightarrow \text{push } 2$
  - $\text{st} = [1, 2]$
- 

$i = 3 \rightarrow h[3] = 6$

- $h[2] = 5 < 6 \rightarrow \text{push } 3$
  - $\text{st} = [1, 2, 3]$
- 

$i = 4 \rightarrow h[4] = 2$

- $h[3] = 6 \geq 2 \rightarrow \text{pop } 3$ 
    - $ht = 6, rt = 4, lt = 2 \rightarrow \text{width} = 4 - 2 - 1 = 1$
    - $\text{area} = 6 \times 1 = 6 \rightarrow \text{maxA} = 6$
  - $h[2] = 5 \geq 2 \rightarrow \text{pop } 2$ 
    - $ht = 5, rt = 4, lt = 1 \rightarrow \text{width} = 4 - 1 - 1 = 2$
    - $\text{area} = 5 \times 2 = 10 \rightarrow \text{maxA} = 10$
  - $h[1] = 1 < 2 \rightarrow \text{stop, push } 4$
  - $\text{st} = [1, 4]$
- 

$i = 5 \rightarrow h[5] = 3$

- $h[4] = 2 < 3 \rightarrow \text{push } 5$
  - $\text{st} = [1, 4, 5]$
- 

$i = 6$  (sentinel, treat  $h[i] = -\infty$ )

Now force pop all remaining bars.

1. Pop 5
  - $ht = 3, rt = 6, lt = 4 \rightarrow width = 6 - 4 - 1 = 1$
  - $area = 3 \times 1 = 3 \rightarrow \max A = 10$
2. Pop 4
  - $ht = 2, rt = 6, lt = 1 \rightarrow width = 6 - 1 - 1 = 4$
  - $area = 2 \times 4 = 8 \rightarrow \max A = 10$
3. Pop 1
  - $ht = 1, rt = 6, lt = -1 \rightarrow width = 6 - (-1) - 1 = 6$
  - $area = 1 \times 6 = 6 \rightarrow \max A = 10$

Stack empty, push 6. Done.

## ◆ Final Answer

$\max A = 10$

This corresponds to rectangle formed by bars  $[5, 6]$  (heights 5 & 6),  $width = 2 \rightarrow area = 10$ .

## 🔥 Problem 3: Maximal Rectangle in Matrix (Hard)

📌 **Statement:** Largest rectangle of 1s in a binary matrix.

**Trick:**

- Treat each row as **histogram of heights**.
- For each row, update heights & apply **largest histogram**.

**Code ( $O(m*n)$ ):**

```
int maximalRectangle(vector<vector<char>>& mat) {
    if(mat.empty()) return 0;
    int m=mat.size(), n=mat[0].size();
    vector<int> h(n,0);
    int ans=0;
    for(int i=0;i<m;i++){
        for(int j=0;j<n;j++){
            h[j] = (mat[i][j]=='1') ? h[j]+1 : 0;
            // apply largestRectangleArea
            stack<int> st;
            for(int k=0;k<=n;k++){
                while(!st.empty() && (k==n || h[st.top()]>=h[k])){
```

```

        int ht=h[st.top()]; st.pop();
        int rt=k, lt=st.empty()? -1:st.top();
        ans=max(ans, ht*(rt-lt-1));
    }
    st.push(k);
}
}
return ans;
}

```

### TC Example

```

matrix =
[["1","0","1","0","0"],
 ["1","0","1","1","1"],
 ["1","1","1","1","1"],
 ["1","0","0","1","0"]]

```

👉 Output = 6

## Problem 4: Sum of Subarray Minimums (Medium)

📌 **Statement:** Return sum of minimum element of all subarrays.

**Trick:**

- Each element is **minimum** in some subarrays  $\rightarrow$  Contribution = val \* left \* right.
- left = i - prevSmallerIndex
- right = nextSmallerIndex - i

**Code (O(n)):**

```

int sumSubarrayMins(vector<int>& arr) {
    int n=arr.size(), mod=1e9+7;
    vector<int> left(n), right(n);
    stack<int> st;
    // prev smaller
    for(int i=0;i<n;i++){
        while(!st.empty() && arr[st.top()]>arr[i]) st.pop();
        left[i]= st.empty()? i+1 : i-st.top();
        st.push(i);
    }
    while(!st.empty()) st.pop();
    // next smaller
    for(int i=n-1;i>=0;i--){
        while(!st.empty() && arr[st.top()]>=arr[i]) st.pop();
        right[i]= st.empty()? n-i : st.top()-i;
        st.push(i);
    }
}

```

```

    long long ans=0;
    for(int i=0;i<n;i++)
        ans=(ans + 1LL*arr[i]*left[i]*right[i])%mod;
    return ans;
}

```

### TC Example

arr = [3,1,2,4]

👉 Subarray mins sum = 17

## 🔥 Problem 5: Sum of Subarray Ranges (Medium)

📌 **Statement:** Return sum of  $(\max - \min)$  for all subarrays.

**Trick:**

- $\text{Sum}(\max) - \text{Sum}(\min)$ .
- Use same technique as above for both max & min.

**Code (O(n)):**

```

long long subArrayRanges(vector<int>& arr) {
    int n=arr.size();
    auto helper=[&](bool isMin){
        vector<int> left(n), right(n); stack<int> st;
        for(int i=0;i<n;i++){
            while(!st.empty() && (isMin? arr[st.top()]>arr[i] :
arr[st.top()]<arr[i])) st.pop();
            left[i]=st.empty()? i+1 : i-st.top();
            st.push(i);
        }
        while(!st.empty()) st.pop();
        for(int i=n-1;i>=0;i--){
            while(!st.empty() && (isMin? arr[st.top()]>=arr[i] :
arr[st.top()]<=arr[i])) st.pop();
            right[i]=st.empty()? n-i : st.top()-i;
            st.push(i);
        }
        long long sum=0;
        for(int i=0;i<n;i++) sum+=1LL*arr[i]*left[i]*right[i];
        return sum;
    };
    return helper(false)-helper(true);
}

```

### TC Example

arr = [1,2,3]

📌 Subarrays: [1],[2],[3],[1,2],[2,3],[1,2,3]  
Ranges =  $0+0+0+1+1+2 = 4$

---

## Mega Trick Recap

- **Histogram / Rectangle** → Use `prev smaller + next smaller`.
- **Water** → Use `min(leftMax, rightMax)`.
- **Sum of Subarray contributions** → Each element contributes via `(i-prev) * (next-i)`.
- **Matrix Rectangle** = row-wise histogram

## 5. Simulation & Game Problems (Stack = Brain)

---

### **1** Asteroid Collision (Medium)

#### Problem Statement:

- You are given an array of integers representing asteroids in a row.
- Positive → moving right, Negative → moving left.
- If two asteroids meet, the smaller explodes. Equal size → both explode.
- Return the state after all collisions.

#### Pattern:

- Use **stack to simulate collisions**.
- Stack stores **moving right asteroids**; when a negative asteroid appears, check collisions.

#### Trick to remember:

- Positive → always push.
- Negative → keep popping **smaller positives**.
- If top positive  $\geq \text{abs}(\text{negative})$ , stop popping.

#### Code (O(n), O(n) space):

```
vector<int> asteroidCollision(vector<int>& ast) {  
    stack<int> st;  
    for(int a : ast) {
```

```

    bool alive = true;
    while(alive && a < 0 && !st.empty() && st.top() > 0){
        if(st.top() < -a) st.pop();
        else if(st.top() == -a){ st.pop(); alive=false; }
        else alive=false;
    }
    if(alive) st.push(a);
}
vector<int> res(st.size());
for(int i=st.size()-1;i>=0;i--){ res[i]=st.top(); st.pop(); }
return res;
}

```

### Test Case:

ast = [5, 10, -5]

Output → [5, 10]

### Variation:

- Multi-directional collision in 2D → stack logic + deque.
- Continuous simulation in games → same pattern.

## 2 Remove K Digits (Medium → Greedy + Monotonic)

### Problem Statement:

- Given a number as string `num` and integer `k`.
- Remove `k` digits to get the **smallest possible number**.

### Pattern:

- **Monotonic increasing stack of digits.**
- Pop top of stack if **current digit < stack top** and `k > 0`.
- Push current digit.

### Trick to remember:

- Always ask: *“Does this new digit allow me to remove bigger digits before it?”* → stack.
- Remove leading zeros at the end.

### Code (O(n), O(n) space):

```

string removeKdigits(string num, int k) {
    string st="";
    for(char c: num){
        while(!st.empty() && k>0 && st.back()>c){ st.pop_back(); k--; }
        st.push_back(c);
    }
}

```

```

    }
    while(k>0){ st.pop_back(); k--; }
    // remove leading zeros
    int i=0; while(i<st.size() && st[i]=='0') i++;
    st=st.substr(i);
    return st.empty()?"0":st;
}

```

#### Test Case:

num = "1432219", k=3

Output → "1219"

#### Variation:

- Max number after removing k digits → reverse comparison `st.back() < c` → >
- Problems like “Lexicographically smallest subsequence” → same stack trick.



## Stack = Brain Trick Summary

1. **Asteroid / Collision / Game Simulation:**
  - Stack stores “active entities”, resolve interaction rules.
2. **Remove / Greedy Problems:**
  - Stack stores “optimal elements so far”, pop if a new element improves global goal.
3. **Always ask:**

Do I need to remove previous elements if a new element is worse/better?



## 6. Sliding Window Problems (Queue / Deque Magic)

### 1 Sliding Window Maximum (Hard → Monotonic Deque)

#### Problem Statement:

- Given an array `arr` and window size `k`, for each sliding window of size `k`, find the **maximum element**.

**Pattern:**

- Use **monotonic decreasing deque** (store indices).
- Front of deque = max of current window.

**Trick to Remember:**

- Maintain **indices of candidates** in deque.
- Pop indices **out of window** from front.
- Pop indices **smaller than current** from back (they can never be max).

**Code (O(n), O(k) space):**

```
vector<int> maxSlidingWindow(vector<int>& arr, int k) {
    int n = arr.size();
    deque<int> dq;
    vector<int> res;
    for(int i=0; i<n; i++){
        if(!dq.empty() && dq.front()==i-k) dq.pop_front();
        while(!dq.empty() && arr[dq.back()] < arr[i]) dq.pop_back();
        dq.push_back(i);
        if(i >= k-1) res.push_back(arr[dq.front()]);
    }
    return res;
}
```

**Test Case:**

arr = [1, 3, -1, -3, 5, 3, 6, 7], k=3

Output → [3, 3, 5, 5, 6, 7]

---

## Variation 1: First Negative in Every Window

**Problem Statement:**

- Given array and window size k, find **first negative number** in each window.

**Pattern:**

- Use **deque to store negative indices**.
- Front = first negative in window.

**Code:**

```
vector<int> firstNegativeInWindow(vector<int>& arr, int k){
    int n = arr.size();
    deque<int> dq;
    vector<int> res;
```

```

for(int i=0;i<n;i++){
    if(!dq.empty() && dq.front()==i-k) dq.pop_front();
    if(arr[i]<0) dq.push_back(i);
    if(i >= k-1){
        res.push_back(dq.empty()?0:arr[dq.front()]);
    }
}
return res;
}

```

#### Test Case:

arr = [12,-1,-7,8,-15,30,16,28], k=3

Output → [-1,-1,-7,-15,-15,0]



## Trick Summary (Sliding Window / Deque)

1. **Max / Min in Window** → **Monotonic Deque** (store indices).
2. **Remove out-of-window indices** from front.
3. **Remove smaller/larger elements** from back.
4. Front of deque → **answer for current window**.
5. **Variations:**
  - First negative / positive in window
  - Max/min temperature in last k days
  - Sum / count with conditions (can adapt deque pattern)



#### Memory Hook:

- Think of deque as a **line of soldiers**, front = strongest.
- New soldier comes → remove weaker soldiers behind.
- Old soldier steps out → pop front.



## 7. Special Stack/Queue Designs (Interview HOT)

### 1 Min Stack (Easy-Medium)

#### Problem Statement:

- Design a stack that supports `push`, `pop`, `top`, and `getMin()` in **O(1) time**.

**Pattern:**

- Use **auxiliary stack to store minimum so far**.

**Trick to Remember:**

- Push into **main stack normally**.
- Push **min(current, top of minStack)** into `minStack`.
- Pop both stacks simultaneously.

**Code (O(1) all ops, O(n) space):**

```
class MinStack {
    stack<int> st, minSt;
public:
    MinStack() {}
    void push(int x) {
        st.push(x);
        if(minSt.empty() || x<=minSt.top()) minSt.push(x);
        else minSt.push(minSt.top());
    }
    void pop(){ st.pop(); minSt.pop(); }
    int top(){ return st.top(); }
    int getMin(){ return minSt.top(); }
};
```

**Test Case:**

`push(5), push(3), push(7), getMin(), pop(), getMin()`

Output → 3, 3

**Variation:**

- Track max instead of min.
- Min Stack using **single stack + encoded values** → advanced O(1) space trick.

## 2 LRU Cache (Hard)

**Problem Statement:**

- Design a **Least Recently Used cache** with `get(key)` and `put(key, value)` **O(1) time**.

**Pattern:**

- Use **Doubly Linked List + HashMap**.
- DLL → stores **recency order**.
- Map → stores **key** → **node**.

#### Trick to Remember:

- Every get/put → move node to **front** (most recent).
- Remove **tail** if over capacity → LRU eviction.

#### Code (O(1) ops, O(capacity) space):

```
class LRUCache {
    int cap;
    list<pair<int,int>> dll;
    unordered_map<int, list<pair<int,int>>::iterator> mp;
public:
    LRUCache(int capacity){ cap=capacity; }
    int get(int key){
        if(mp.find(key)==mp.end()) return -1;
        auto it=mp[key]; int val=it->second;
        dll.erase(it);
        dll.push_front({key,val});
        mp[key]=dll.begin();
        return val;
    }
    void put(int key,int val){
        if(mp.find(key)!=mp.end()){ dll.erase(mp[key]); }
        dll.push_front({key,val});
        mp[key]=dll.begin();
        if(dll.size()>cap){
            mp.erase(dll.back().first);
            dll.pop_back();
        }
    }
};
```

#### Test Case:

```
LRUCache(2)
put(1,1), put(2,2), get(1), put(3,3), get(2)
```

Output → 1, -1

#### Variation:

- MRU (Most Recently Used) Cache → remove **head** instead of tail.

## 3 LFU Cache (Hard → Advanced Frequency Map + DLL)

## Problem Statement:

- Design a **Least Frequently Used cache** with `get(key)` and `put(key, value)` **O(1)**.

## Pattern:

- **Freq Map:** `freq`  $\rightarrow$  DLL of keys with same frequency.
- **HashMap:** `key`  $\rightarrow$  (value, freq, node pointer)

## Trick to Remember:

- On `get`  $\rightarrow$  increment `freq`  $\rightarrow$  move node to next `freq` DLL.
- On eviction  $\rightarrow$  remove **least freq & oldest node**.

## Code Concept (O(1) ops, complex, simplified skeleton):

```
struct Node{
    int key, val, freq;
    Node(int k, int v) {key=k, val=v, freq=1;}
};

class LFUCache {
    int cap, minf;
    unordered_map<int, Node*> keyNode;
    unordered_map<int, list<Node*>> freqList;
public:
    LFUCache(int capacity) { cap=capacity; minf=0; }
    int get(int key) {
        if (keyNode.find(key) == keyNode.end()) return -1;
        Node* n = keyNode[key];
        freqList[n->freq].remove(n);
        if (freqList[n->freq].empty() && n->freq == minf) minf++;
        n->freq++;
        freqList[n->freq].push_front(n);
        return n->val;
    }
    void put(int key, int value) {
        if (cap == 0) return;
        if (keyNode.find(key) != keyNode.end()) {
            keyNode[key]->val = value;
            get(key);
            return;
        }
        if (keyNode.size() == cap) {
            Node* toDel = freqList[minf].back();
            freqList[minf].pop_back();
            keyNode.erase(toDel->key);
            delete toDel;
        }
        Node* n = new Node(key, value);
        keyNode[key] = n;
        freqList[1].push_front(n);
        minf = 1;
    }
};
```

### Test Case:

```
LFUCache(2)  
put(1,1), put(2,2), get(1), put(3,3), get(2), get(3)
```

Output  $\rightarrow 1, -1, 3$

### Variation:

- LRU = LFU with freq=1.
- Advanced caches in **databases, OS, CPU** use same logic.

---

## Trick Summary (Special Stack/Queue Designs)

1. **Min Stack**  $\rightarrow$  maintain min so far
2. **LRU/LFU**  $\rightarrow O(1)$  ops  $\rightarrow$  HashMap + DLL
3. Always ask:

Can I simulate recency/frequency or min/max using stack/DLL/hashmap combo?

---

## 8. Matrix/Graph + Stack/Queue (Advanced Mix)

---

### **1** The Celebrity Problem (Hard $\rightarrow$ Matrix + Stack)

#### Problem Statement:

- There are  $n$  people at a party.
- Celebrity = known by everyone but knows no one.
- Given  $M[i][j] = 1$  if  $i$  knows  $j$ , else 0. Find the celebrity.

#### Pattern:

- Use **stack to eliminate non-celebrities**.

### Trick to Remember:

- Push all candidates onto stack.
- **Pop 2 people at a time:**
  - If A knows B  $\rightarrow$  A cannot be celebrity  $\rightarrow$  push B back
  - Else  $\rightarrow$  B cannot be celebrity  $\rightarrow$  push A back
- Check final candidate by verifying **row = 0, column = n-1**

### Code (O(n) time, O(n) space):

```
int celebrity(vector<vector<int>>& M, int n){
    stack<int> st;
    for(int i=0;i<n;i++) st.push(i);
    while(st.size()>1){
        int A=st.top(); st.pop();
        int B=st.top(); st.pop();
        if(M[A][B]==1) st.push(B); // A knows B  $\rightarrow$  A not celebrity
        else st.push(A);           // A doesn't know B  $\rightarrow$  B not celebrity
    }
    int c=st.top();
    for(int i=0;i<n;i++){
        if(i!=c && (M[c][i]==1 || M[i][c]==0)) return -1;
    }
    return c;
}
```

### Test Case:

```
M = [[0,1,0],
      [0,0,0],
      [0,1,0]], n=3
```

Output  $\rightarrow$  1

### Variation:

- Find multiple celebrities  $\rightarrow$  same elimination pattern.
- Social network influence ranking  $\rightarrow$  matrix + stack logic.

---

## 2 DFS Iterative with Stack (Concept)

### Problem Statement:

- Standard DFS traversal on graph using **stack** (instead of recursion).

### Pattern:

- Push source node onto stack.
- Pop node → mark visited → push unvisited neighbors.

**Code ( $O(V+E)$ ,  $O(V)$  space):**

```
void dfsIterative(int V, vector<int> adj[]){
    vector<bool> visited(V, false);
    stack<int> st;
    st.push(0); // starting node
    while(!st.empty()){
        int u=st.top(); st.pop();
        if(!visited[u]){
            visited[u]=true;
            cout<<u<<" ";
            for(int v: adj[u])
                if(!visited[v]) st.push(v);
        }
    }
}
```

**Test Case:**

Graph: 0→1, 0→2, 1→2

Output (DFS): 0 2 1 (one possible)

**Variation:**

- Topological sort → DFS + stack logic.
- Connected components → iterative DFS.

## 3 BFS with Queue (Concept)

**Problem Statement:**

- Standard BFS traversal on graph using **queue**.

**Pattern:**

- Push source node into queue.
- Pop node → mark visited → push unvisited neighbors.

**Code ( $O(V+E)$ ,  $O(V)$  space):**

```
void bfs(int V, vector<int> adj[]){
    vector<bool> visited(V, false);
    queue<int> q;
    q.push(0);
```

```

visited[0]=true;
while(!q.empty()){
    int u=q.front(); q.pop();
    cout<<u<<" ";
    for(int v: adj[u]){
        if(!visited[v]){
            visited[v]=true;
            q.push(v);
        }
    }
}
}

```

### Test Case:

Graph: 0->1, 0->2, 1->2  
Output (BFS): 0 1 2

### Variation:

- Shortest path in unweighted graph → BFS
- Level order traversal of tree → BFS

## Trick Summary (Matrix/Graph + Stack/Queue)

1. **Celebrity** → **elimination using stack** →  $O(n)$
2. **DFS** → **iterative using stack**
3. **BFS** → **iterative using queue**
4. Always ask:

Can I simulate traversal / elimination / ordering using stack or queue efficiently?